BAHAN AJAR

SOFTWARE SIMULATOR Z80 IDE

DIKLAT MIKROPROSESOR Z80

PUSAT PENGEMBANGAN PEMBERDAYAAN PENDIDIK DAN TENAGA PENDIDIK

BIDANG MESIN DAN TEKNIK INDUSTRI

2016

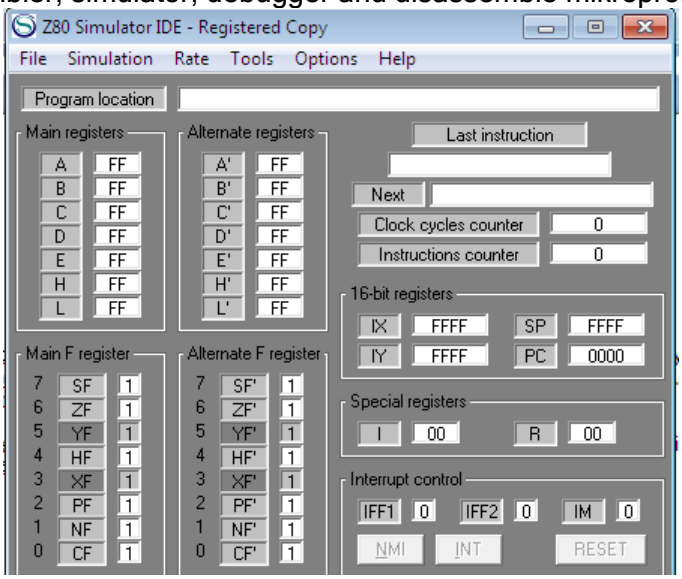
DAFTAR ISI

DAFTAR ISI 2
ANTARMUKA Z80 SIMULATOR IDE 3
Informasi umum 3
Menu file 3
- clear memory 3
- load program 3
- save memory 3
menu simulasi 4
- start 4
- step 4
- stop 4
Menu rate 4
tools menu 4
- memory editor 4
- disassembler 5
- peripheral devices 5
- i/o ports editor 6
- breakpoints manager 7
-Simulasi Log View 7
-Interupsi 8
Aktifitas Memulai Menggunakan Software Simulator Z80 IDE 9
A. Membuat Program Assembler 9
B. Menjalankan Simulasi 10
C. Melihat Memori Editor 11
D. Melihat I/O Port 12
E. Melihat Simulasi Peripheral 13
F. Menggunakan Break Points Manager 14
LATIHAN 16

ANTARMUKA Z80 SIMULATOR IDE

Informasi umum

z80 simulator ide adalah aplikasi yang mendukung pembelajaran dan pengembangan z80 secara grafis yang muda digunakan dan bekerja di aplikasi window yang terintegrasi dengan basic compiler, assembler, simulator, debugger and disassemble mikroprosesor 8 bit zilog z80



Gambar 1Tampilan Simulator Z80

Tampilan aplikasi utama menunjukkan register internal dan struktur flag dari mikroprosesor z80, mnemonic dari instruksi yang telah, sedang dan akan dieksekusi. siklus klok instruksi counter dan antarmuka inrturpsi.

Menu file

- clear memory

perintah ini mengembalikan keadaan awal dari memory64k dengan alamat antara 0000h-ffffh dengan asumsi telah dipasang pada cpu. memori diisi dengan kode 00h

- load program

perintah ini mengisikan file program kedalam memori cpu. file program ini harus dalam format intel hex atau binary (berakhiran .obj) dari memori yang dimulai dari alamat 0000h sebanyak 64k.

- save memory

perintah ini menyimpan isi memori ke sebuah file

menu simulasi

- start

z80 simulator ide masuk mode simulasi dan mengeksekusi instruksi mulai dari lokasi memori yang sesuai dengan parameter alamat awal. nilai bawaan untuk parameter ini adalah 0000h dan nilai ini dapat diubah menggunakan perintah change starting address pada menu option

- step

perintah ini akan aktif hanya ketika memilih mode simulasi step by step.
instruksi next dijalankan padasetiap menekan tombol f2

- stop

z80 simulator ide keluar dari mode simulasi dan menampilkan informasi tentang jumlah keseluruhan instruksi yang dieksekusi, durasi simulasi dan sebenarnya dari jumlah klok simulasi.

Menu rate

pengguna dapat mengubah kecepatan simulasi. ini dapat diatur selama simulasi juga.

- step by step

instruksi secara satu per satu akan dijalankan dengan menekan tombol step

- slow

selang waktu 1500 ms.

- normal

selang waktu 250 ms.

- fast

selang waktu 50 ms.

- extremely fast

sangat singkat dan tergantung kecepatan komputer

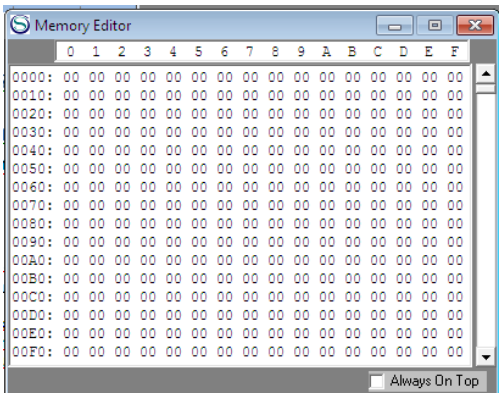
- ultimate

jendela simulator utama tidak terus menerus direpresh setiap instruksi di simulasikan yang secara signifikan memperbaiki kinerja simulasi

tools menu

- memory editor

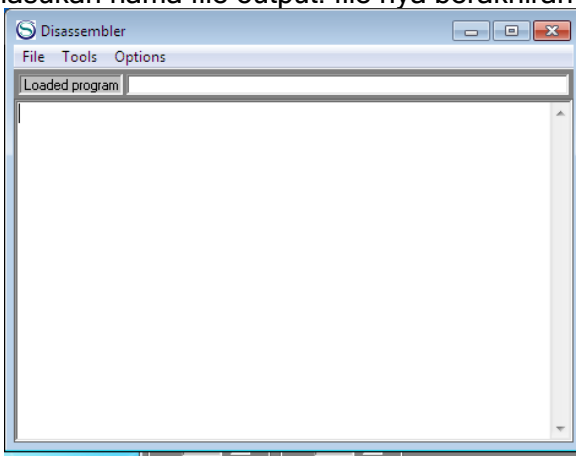
memori editor dapat digunakan untuk melihat isi memori z80 sebesar 64k



Gambar 2 Memori Editor

- disassembler

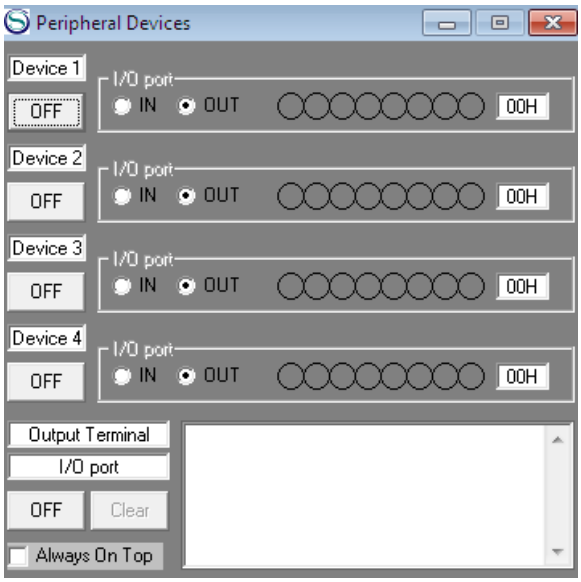
z80 simulator ide memiliki disassembler serba bisa dengan dimulai dari menekan tombol ini. disassembler sebenarnya terpisah dari simulator dan memiliki memori program sendiri. sehingga perlu mengisi file program (intel hex file atau binary image) kedalam memori disassembler dulu. disassembler akan selalu mulai dari alamat 0000h. setelah proses selesai disassembler akan menampilkan hasil file listing. listing yang dihasilkan dapat disimpan pada disk. pengguna akan memasukan nama file output. file nya berakhiran lst.



Gambar 3 Tampilan Disasembler

- peripheral devices

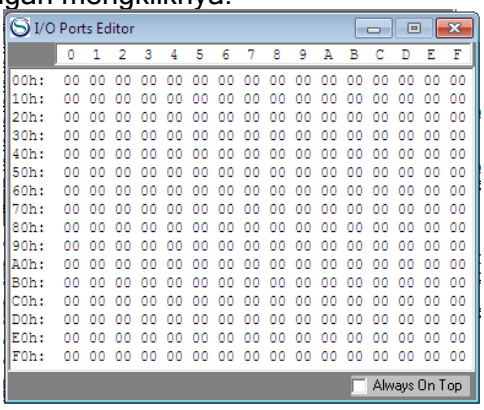
alat ini berguna untuk mengontrol perintal in dan out. sampai 4 komponen peripheral i/o dapat disetup dan ada juga satu terminal output yang berguna untuk melihatkarakter ascii yang dikirim ke salah satu port. byte yang dikirim dengan instruksi outditampilkan secara grafis masing-masing bit. jika alat dikonfigurasikan sebagai peralatan input nilainya dapat diset dengan memasukan secara langsung.



Gambar 4 Peralatan Peripheral

- i/o ports editor

bagian ini mengakses antarmuka grafis untuk mengakses kontrrol port i/o sampai lengkap. nilai pada port io dapat diubah dengan mengkliknya.



Gambar 5 I/O PORT EDITOR

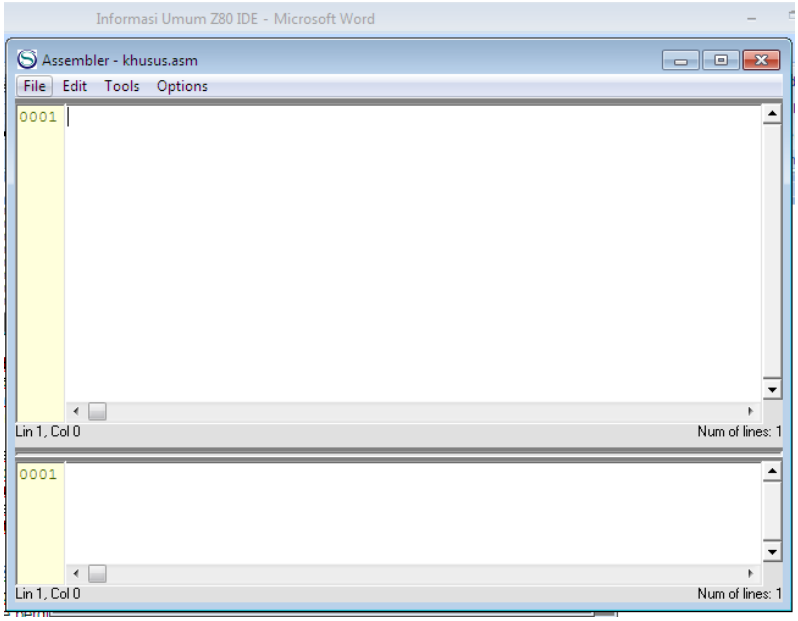
- external modules

alat ini seharusnya dapat digunakan untuk mendirikan antarmuka otomasi dengan sejumlah maksimal 5 modul luar. hal ini dibutuhkan untuk masuk ke nama kelas yang diberikan oleh peralatan luar pada bentuk nama aplikas, nama objek agar terjadi koneksidengan nya.

aplikasi client/server akan dimulai dan diakhiri secara otomatis oleh applications z80 simulator ide.

- assembler

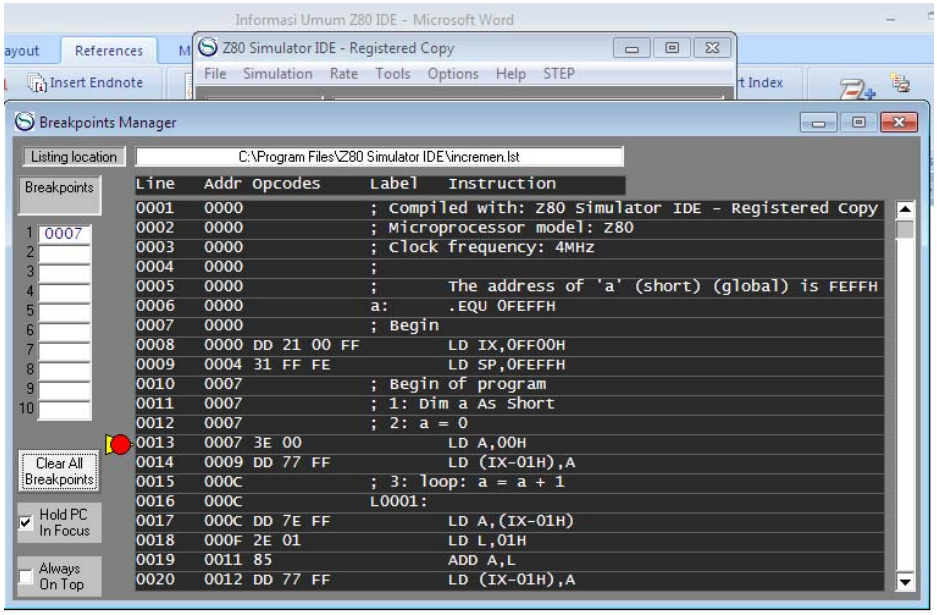
perintah ini mulai mengintegrasikan bahasa assembler. file sumber assembler dapat diedit, langsung diasembly dan akhirnya diisikan kedalam memori pada lingkungan grafik yang sama. ektensi file nya adalah asm. setelah proses assembly berhasil dua file baru dihasilkan yang satu dengan berakhiran obj yaitu binary image yang dapat secara langsung diisikan ke memori cpu dan satunya lagi berakhiran lst yang terdiri dari assembler listing. jika opsi file hex dipilih akan dihasilkan pula file hex dalam format intel hex.



Gambar 1 Area Kerja Assembler

- breakpoints manager

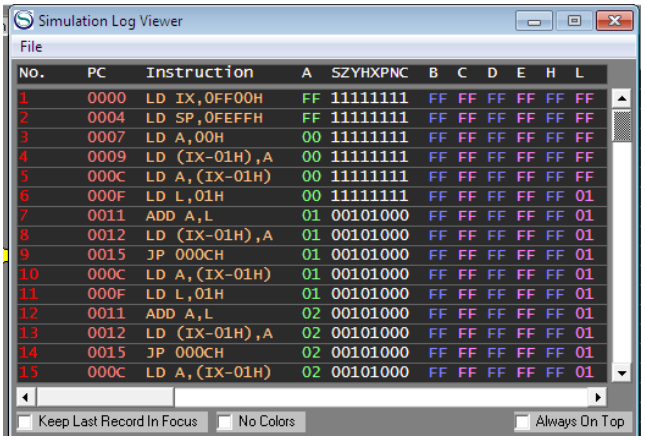
perintah ini dapat digunakan untuk debug dan monitor eksekusi program.if listing file assembler program pada memori tidak ada listing alternative berguna akan diperoleh dari internal disasembler.



Gambar 6 Breakpoint Manager

-Simulasi Log View

Digunakan untuk menampilkan Instruksi yang sedang disimulasikan



No.	PC	Instruction	A	SZYHXPNC	B	C	D	E	H	L
1	0000	LD IX,0FF00H	FF	11111111	FF	FF	FF	FF	FF	FF
2	0004	LD SP,0FEFFH	FF	11111111	FF	FF	FF	FF	FF	FF
3	0007	LD A,00H	00	11111111	FF	FF	FF	FF	FF	FF
4	0009	LD (IX-01H),A	00	11111111	FF	FF	FF	FF	FF	FF
5	000C	LD A,(IX-01H)	00	11111111	FF	FF	FF	FF	FF	FF
6	000F	LD L,01H	00	11111111	FF	FF	FF	FF	FF	01
7	0011	ADD A,L	01	00101000	FF	FF	FF	FF	FF	01
8	0012	LD (IX-01H),A	01	00101000	FF	FF	FF	FF	FF	01
9	0015	JP 000CH	01	00101000	FF	FF	FF	FF	FF	01
10	000C	LD A,(IX-01H)	01	00101000	FF	FF	FF	FF	FF	01
11	000F	LD L,01H	01	00101000	FF	FF	FF	FF	FF	01
12	0011	ADD A,L	02	00101000	FF	FF	FF	FF	FF	01
13	0012	LD (IX-01H),A	02	00101000	FF	FF	FF	FF	FF	01
14	0015	JP 000CH	02	00101000	FF	FF	FF	FF	FF	01
15	000C	LD A,(IX-01H)	02	00101000	FF	FF	FF	FF	FF	01

Gambar 2 Simulasi Log View

-Interupsi

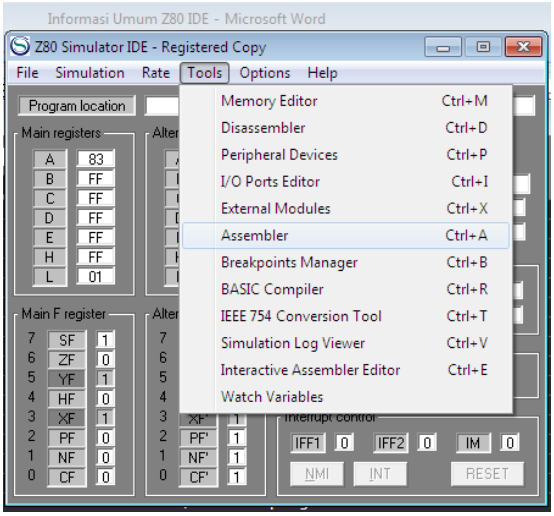
Semua interupsi dimungkinkan untuk disimulasikan tombol nmi untuk interupsi non maskable Interupsi. Tombol INT untuk menteriger Interupsi maskable. Untuk mengembalikan dari interupsi tekan tombol Reset

Aktifitas Memulai Menggunakan Software Simulator Z80 IDE

A. Membuat Program Assembler

Langkah-langkah :

1. Pilih Menu "**Tool**"→ **Assembler**

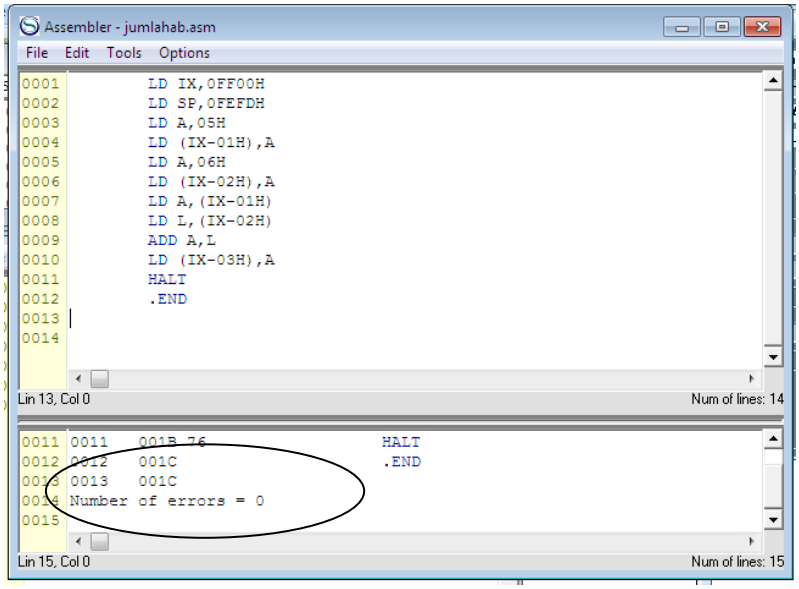


Gambar 3 Memilih Asembler

2. Pilih File → New
3. Buatlah Program seperti berikut ini :
   Program menjumlahkan dua buah bilangan pada alamat FEFFH dan FEFEH hasilnya disimpan pada FEFDH

```
LD IX,0FF00H
LD SP,0FEFDH
LD A,05H
LD (IX-01H),A
LD A,06H
LD (IX-02H),A
LD A,(IX-01H)
LD L,(IX-02H)
ADD A,L
LD (IX-03H),A
HALT
.END
```

4. Simpan File dengan nama penjumlahan_ab.asm
5. Kemdian untuk mengkompail menjadi file Hex atau obj pilih Tool → Assemble &Load lakukan sampai tidak ada error. Jika ada error periksa program dan perbaiki.



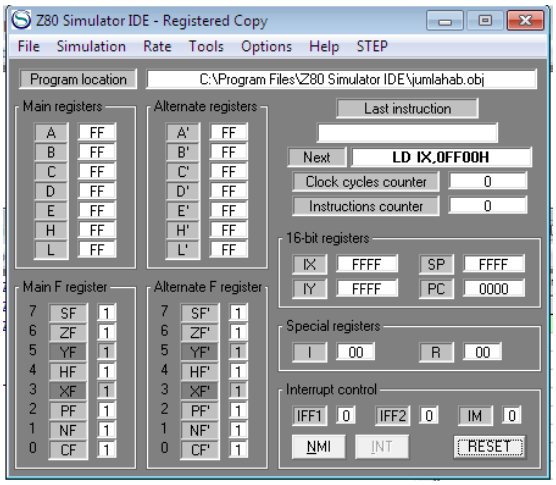
Gambar 7 Indikator Tidak Ada Kesalahan

6. Selesai sekarang file hex sudah tersedia

B. Menjalankan Simulasi

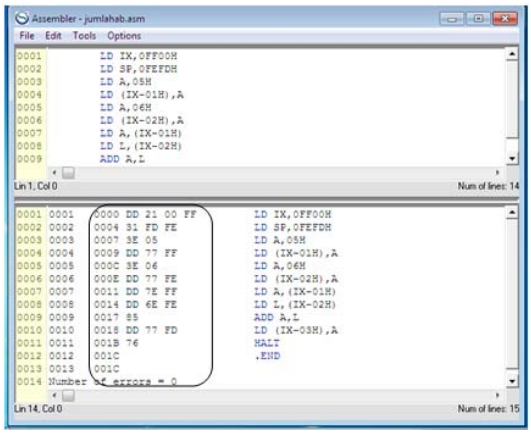
Langkah-langkah :

1. Ambil File image berakhiran obj (penjumlahan_ab.obj) dengan cara File →Load Program dan cari file penjumlahan_ab.obj kemudian tekan tombol Open
2. Sekarang akan mencoba simulasi melalui rate step by step caranya adalah Pilih menu **Rate →step by Step**
3. Pilih **Simulation→Start** setelah dijalankan dengan menekan tombol start akan munculmenu baru yaitu STEP. Anda tingga menekan tombol **step** dan akan muncul Instruksi yang sedang dieksekusi dalam bentuk bahasa assembly serta ada instruksi terakhir dan berikutnya.
4. Dengan Step by Step Anda dapat melihat perubahan register untuk masing-masing step melalui memori editor.



Gambar 8 Simulasi Dengan Strep by Step

5. Instruksi yang akan dieksekusi adalah LD IX,0ff00h, Amati nilai IX sekarang……. Tekang tombol **STEP.** Amati kembali nilai **IX =…….**
6. Hentikan Simulasi dengan cara **Simulation → Stop**
7. Selain itu anda dapat melihat kode operasi (OPCODE) yang berada pada jendela paling bawah jika program berhasil dikompail.



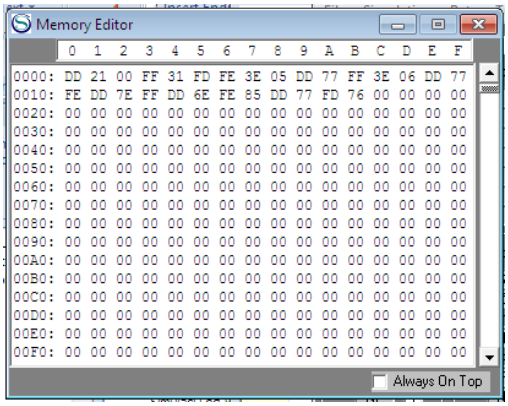
C. Melihat Memori Editor

Anda Dapat melihat memori sebanyak 64K dari 0000H s.d 0FFFFH melalui memori editor. Memori ini terdiri dari memori program maupun Variabel yang digunakan.

Langkah-langkah :

Klik **Tools** → Memori **Editor**. Dan akan tampak seperti pada gambar dibawah ini :

Alamat 0000H s.d. 001BH diisi memori program.



Gambar 9 Memori Editor

8. **Melalui memori editor dan melalui register Isilah table berikut ini :**

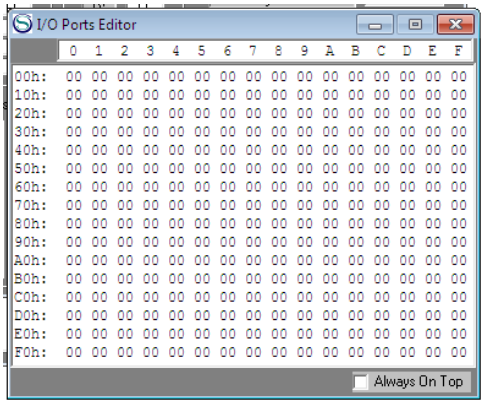
No	Instruksi	Nilai Register yang berubah
1	LD IX,0ff00h	IX =0ffffh menjadi IX =0ff00h
2	LD IX,0FF00H	……
3	LD SP,0FEFDH	……
4	LD A,05H	……
5	LD (IX-01H),A	……
6	LD A,06H	……
7	LD (IX-02H),A	……
8	LD A,(IX-01H)	……
9	LD L,(IX-02H)	……
10	ADD A,L	……
11	LD (IX-03H),A	……
12	HALT	……
	.END	

9. Hasil dari operasi berikut dapat dilihat pada memori editor

No	Alamat Memori (Variabel)	Nilai	FUNGSI
1	0FEFFH		A
2	0FEFEH		B
3	0FEFDH		C=A+B

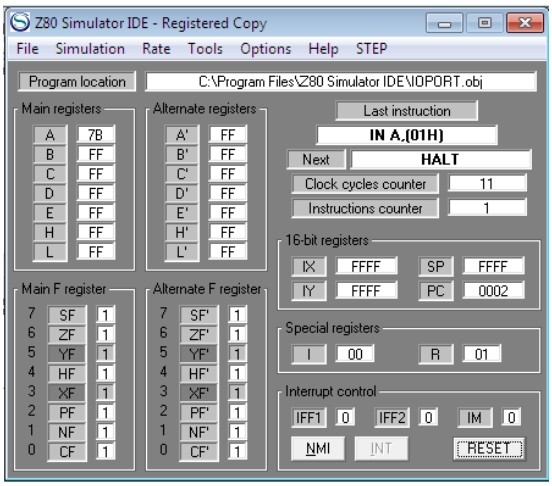
D. Melihat I/O Port

Pada Z80 Anda memungkinkan dapat menggunakan sebanyak 256 Input Output.



Langkah-langkah :

1. Buatlah Instrruksi Program berikut ini :
   Jika Isi dari akumulator adalah 23H dan data pada peralatan luar yang telah dipetakan pada alamat port I/O 01h adalah 7BH
   .org 0000h
   IN A, (01H)
   HALT
2. Lakukan Kompilasi seperti pada kegiatan membuat program sebelumnya.
3. Lakukan Simulasi melalui Step by Step
4. Buka I/O Port Editor
   Amati Port 01H =……..
5. Amati Port Yang digunakan
   Amati Register A =……



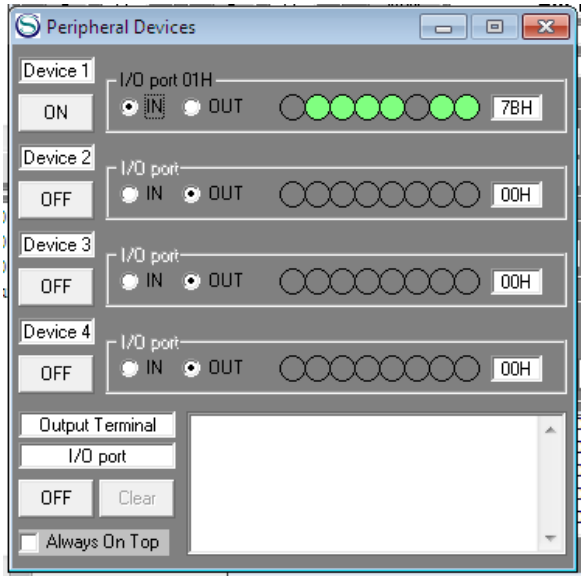
Gambar 10 Register A

E. Melihat Simulasi Peripheral

Pada Peripheral Device Anda dapat melihat tampilan dari Port I/O Secara grafis dalam bentuk Simulasi Lampu LED.

Langkah-langkah :

1. Pilih Tools →Peripheral Devices
2. Klik Device yang mau digunakan Untuk mengaktifkan tekan tombol OFF menjadi ON
3. Tentukan Alamat Port nya contoh 01H
4. Pilih Port tersebut sebagai input atau output. Seperti tampak pada gambar dibawah ini.



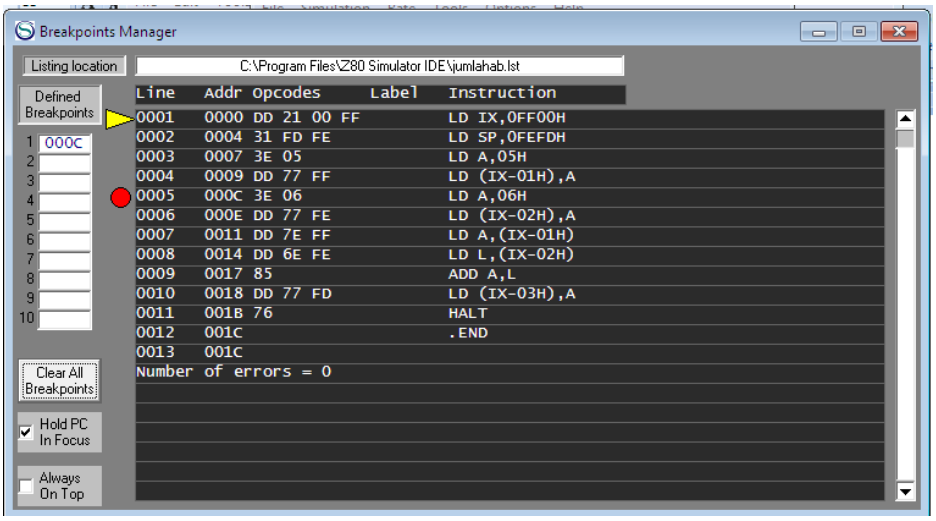
Gambar 11 Peripheral Devices

F. Menggunakan Break Points Manager

Perintah ini dapat digunakan untuk debug dan monitor eksekusi program.if listing file assembler program pada memori tidak ada listing alternative berguna akan diperoleh dari internal disasembler.

Langkah-langkah

1. Ambil Program yang akan disimulasikan dengan Cara File → Load Program
2. Atur **Rate** menjadi Normal. Pilih **Rate→ Normal**
3. Pilih **Tools→ Breakpoints Manager**



Gambar 12 Break Point

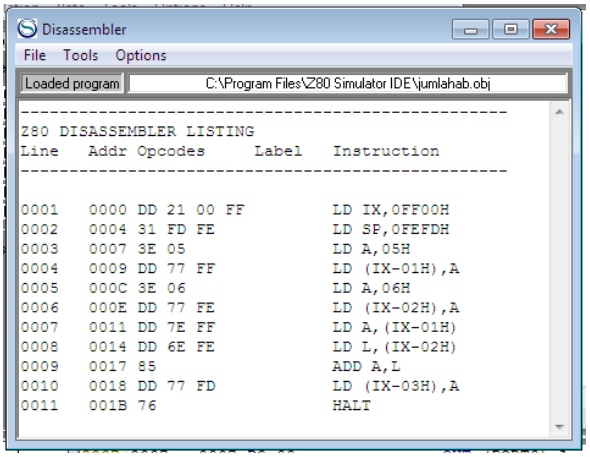
4. Klik Pada line 5 atau alamat 000c sehingga akan mendefinisikan alamat breakpoint pada 000c
5. Jalankan program dengan menekan **Simulation → Start.** Program akan berjalan terus menerus sampai berhenti pada alamat yang dijadikan breakpoint dan rate akan berubah menjadi Step by step
6. Untuk melanjutkan program tekan tombol **Clear breakpoint** dan atur kembali rate menjadi normal atau Fast.

G. Disassembler Program

Fasilitas ini bertujuan untuk mengembalikan file Assembly dari file image (.obj) atau file hex

Langkah-langkah :

1. Pilih Tools→Disassembler



Gambar 13 Disassembler

2. Pilih **File → Load program**
3. Pilih File obj sebagai contoh jumlahab.obj kemudian tekan tombol Open
4. Kemudian Pilih **Tools→Disassemble**
5. Hasilnya akan tampak seperti diatas yang terdiri dari Addr, Opcodes, Label dan Instructions

LATIHAN

1. Buatlah Program berikut ini dan jalankan Simulasi . Tampilkan Register dan Memori yang berubah dan Bagaimana Hasilnya

```
        JP      1800h
        ORG     1800h
        LD      HL,1900H
        LD      A,1H
        LD      B,20H
LOOP    LD      (HL),A
        INC     A
        INC     HL
        DJNZ    LOOP
        HALT
```

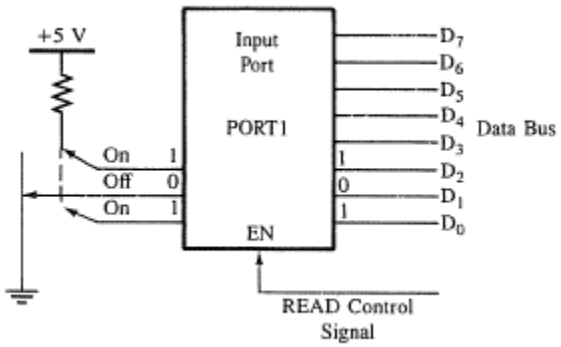
Isilah Tabel berikut ini

INSTRUKSI	Register Yang berubah
JP 1800h	…….
ORG 1800h	……
LD HL,1900H	…….
LD B,20H	…….
LOOP LD (HL),A	…….
INC HL	…….
DJNZ LOOP	…….
HALT	…….

Amati hasil Program tersebut pada memori Editor

MEMORI	Nilai Memori
1900 s.d. 191FH	……,……,……..,……,……,……,……,……, ……,……,……..,……,……,……,……,……,

2. Buatlah Program berikut ini dan jalankan Simulasi . Tampilkan Register dan Memori yang berubah dan Bagimana Hasilnya. Port input terpasang pada 3 bit terendah.



Program :

```
PORT0: EQU    00H
INBUF: EQU    0FF00H
PORT1: EQU    01H
       IN A,  (PORT1)
       AND    07H
       LD     (INBUF),A
       OUT    (PORT0),A
       HALT
```

INSTRUKSI	Register Yang berubah
IN A, (PORT1) AND 07H LD (INBUF),A OUT (PORT0),A HALT	……. …… …… ……

Amati hasil Program tersebut pada I/O Editor dan peripheral

MEMORI	Nilai Memori
Port0 Port1	……. …….